

EE789:Algorithmic Design of Digital Systems

Assignment 2 : Problem 3 Report

Name of student : Prathmesh Baral

Roll no : 23B1260

April 25, 2026

1 Problem Statement

Use all three optimizations: loop-unrolling, word-parallelism, and blocking and measure the improvement in performance relative to the base line For $N=64$.

2 Matrix Multiplication Optimizations

2.1 Base Implementation

The base implementation performs matrix multiplication sequentially using a standard triple-nested loop. Matrices A , B , and $RESULT$ are stored as two-dimensional arrays of 8-bit integers.

Each element $RESULT[I][J]$ is computed as the dot-product of the I^{th} row of A and the J^{th} column of B , implemented in the `dotp` module. The computation iterates over index K , performing one multiplication per cycle and accumulating results in a single variable (SUM).

The top-level `mmul` module computes all (I, J) entries. Since only one multiplication is done per cycle, this serves as a baseline for performance comparison.

Tick count: 2,670,785

2.2 Optimization 1: Loop Unrolling (Factor 2)

Loop unrolling improves performance by executing two multiplications per iteration instead of one. The inner loop is unrolled by a factor of 2 in the `dotp_unroll_2` module.

Two accumulators (SUM0, SUM1) are used to compute:

- $A[I][K] \cdot B[K][J]$
- $A[I][K+1] \cdot B[K+1][J]$

The loop index increments by 2, reducing iterations by half and improving pipeline utilization. This increases throughput through instruction-level parallelism.

Tick count: 1,749,185

2.3 Optimization 2: Word-Level Parallelism

This optimization packs data into 64-bit words, each containing 8 elements. Matrix A is stored in row-major form and B in column-major form to improve access patterns.

A `dot8` module computes 8 multiplications in parallel by splitting each 64-bit input into eight 8-bit values and summing them using a tree structure.

The loop now iterates with stride 8, reducing iterations from N to $N/8$. This improves performance by:

- Reducing memory accesses
- Exploiting data-level parallelism
- Lowering loop overhead

Tick count: 647,361

2.4 Optimization 3: Blocking and Tiling

Blocking divides matrices into four sub-matrices of size $\frac{N}{2} \times \frac{N}{2}$. This allows computing four output elements simultaneously using smaller blocks.

The computation is restructured as:

- $R[I][J] = A_0 \cdot B_0 + A_1 \cdot B_2$
- $R[I][J + N/2] = A_0 \cdot B_1 + A_1 \cdot B_3$
- $R[I + N/2][J] = A_2 \cdot B_0 + A_3 \cdot B_2$
- $R[I + N/2][J + N/2] = A_2 \cdot B_1 + A_3 \cdot B_3$

Four parallel dot-product modules (`dotp0-dotp3`) are used. Loop bounds reduce to $N/2$, improving locality and reducing overhead.

Tick count: 442,465

3 Using All Optimisations

The final implementation combines all three optimisations — loop unrolling, word-level parallelism, and blocking/tiling — into a single unified design, achieving the best possible performance for $N = 64$.

3.1 Overview

The matrices A and B are each divided into four quadrant sub-matrices (A_0, A_1, A_2, A_3 and B_0, B_1, B_2, B_3) of size $\frac{N}{2} \times \frac{N}{16}$, where the second dimension reflects packing: each 64-bit word stores 8 contiguous 8-bit elements. This layout simultaneously exploits blocking (cache locality and reduced loop overhead) and word-level parallelism (8 multiplications per memory access).

3.2 Combined Optimisation Strategy

The `mmul` module iterates I and J from 0 to $\frac{N}{2} - 1$, and at each (I, J) pair, calls four dot-product modules in sequence — `dotp0` through `dotp3` — writing results into all four quadrants of the output matrix simultaneously:

- `dotp0`: computes $R[I][J]$ using sub-matrices A_0, A_1, B_0, B_2
- `dotp1`: computes $R[I][J + N/2]$ using sub-matrices A_0, A_1, B_1, B_3
- `dotp2`: computes $R[I + N/2][J]$ using sub-matrices A_2, A_3, B_0, B_2
- `dotp3`: computes $R[I + N/2][J + N/2]$ using sub-matrices A_2, A_3, B_1, B_3

Each `dotpX` module internally applies **both** loop unrolling (factor 2) and word-level parallelism. The inner loop increments K by 2 per iteration (unrolling), and at each step calls the pipelined `dot8` operator twice — once for K and once for $K + 1$. Each `dot8` call computes 8 multiply-accumulate operations in parallel using an adder tree, reducing partial sums in $\mathcal{O}(\log_2 8) = 3$ levels of addition.

Four independent accumulators (SUM0–SUM3) collect the partial results from each pair of `dot8` calls, and are summed at the end to produce the final 8-bit output. The inner loop pipeline is declared with ``,` allowing successive iterations to overlap and keeping the hardware fully utilised.

3.3 Sources of Speedup

The combined design extracts parallelism at three levels simultaneously:

- **Data-level parallelism:** Each `dot8` call processes 8 elements per cycle, reducing the effective inner loop trip count from N (baseline) to $\frac{N}{16}$.
- **Instruction-level parallelism:** Loop unrolling by a factor of 2 halves the number of loop-control instructions and enables the pipeline to remain full across iterations.
- **Task-level parallelism:** Blocking exposes four independent output elements per outer-loop iteration, computed by four concurrent dot-product modules, cutting the outer loop trip count by a factor of 4.

3.4 Tick Count

Tick count: 65,633 (Speedup: $40.7\times$ over baseline)

3.5 Code Reference

The implementation is listed in Section 5.

Worked examples are presented in Section 6.

The source code bundle is stored in `AA_code_bundle`.

The testbench and corresponding log files are available in `Testing_files`.

```
RESULT[63][60] = 123
RESULT[63][61] = 124
RESULT[63][62] = 125
RESULT[63][63] = 126
[INFO] Results fetched successfully.
[INFO] Computation finished. Ticks = 65633

----- MATRIX A -----
  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16
47  48  49  50  51  52  53  54  55  56  57  58  59  60  61  62  63
  1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
```

Figure 1: Tick count for the implementation using all optimisations.

4 Performance Comparison and Conclusion

The performance of matrix multiplication was evaluated for the base implementation and three successive optimizations. The execution time was measured in terms of clock ticks for $N = 64$.

Implementation	Ticks	Speedup (w.r.t Base)
Base Implementation	2,670,785	1.00×
Optimization 1 (Unroll $\times 2$)	1,749,185	1.53×
Optimization 2 (Packing + dot8)	647,361	4.13×
Optimization 3 (Blocking/Tiling)	442,465	6.03×
All Optimization	65633	40.7×

- **Loop unrolling alone** provides a modest 1.53× speedup by reducing loop-control overhead and enabling two multiplications per iteration. However, it does not change the fundamental memory access pattern or exploit data-level parallelism, so its impact is limited.
- **Word-level parallelism** delivers the largest single-optimisation gain (4.13×), as packing 8 elements into a 64-bit word and computing them simultaneously via the `dot8` operator reduces the inner loop trip count by a factor of 8. The reorganisation of B into column-major packed form is essential to making this access pattern efficient.
- **Blocking/tiling** further improves performance to 6.03× by computing four output elements per outer-loop iteration, reducing loop overhead and improving data reuse. The four parallel `dotp` modules each handle an independent quadrant of the result matrix, exploiting task-level parallelism.
- **Combining all three optimisations** yields a 40.7× speedup — significantly greater than the product of the individual gains would naively suggest. This super-linear benefit arises because the optimisations are synergistic: blocking creates independent sub-problems for each `dotp` module; word parallelism reduces the inner loop iterations within each sub-problem; and loop unrolling keeps the deep pipeline continuously fed, eliminating stall cycles.
- **Diminishing returns** are visible in the individual optimisations: going from Optimisation 2 to Optimisation 3 alone adds only $\sim 1.5\times$ improvement. The dramatic final jump demonstrates that the full benefit of pipelining is only realised when all three techniques are applied together, since each one addresses a different bottleneck (computation, memory bandwidth, and control flow, respectively).

5 Implementaion Using all optimisations

```
$parameter N          64
$pipe GLOBAL_COUNTER: $uint<32> $signal
$storage A0 A1 A2 A3 : $array [{N/2}][{N/16}] $of $uint<64>
$storage B0 B1 B2 B3 : $array [{N/2}][{N/16}] $of $uint<64>
$storage Result : $array [N][N] $of $int<8>

// to initialize the matrices..
$module [init_A0_entry]
    $in (I J: $uint<32> V: $uint<64>) $out () $is
{
    A0[I][J] := ($bitcast ($uint<64>) V)
}

$module [init_A1_entry]
    $in (I J: $uint<32> V: $uint<64>) $out () $is
{
    A1[I][J] := ($bitcast ($uint<64>) V)
}

$module [init_A2_entry]
    $in (I J: $uint<32> V: $uint<64>) $out () $is
{
    A2[I][J] := ($bitcast ($uint<64>) V)
}

$module [init_A3_entry]
    $in (I J: $uint<32> V: $uint<64>) $out () $is
{
    A3[I][J] := ($bitcast ($uint<64>) V)
}

$module [init_B0_entry]
    $in (I J: $uint<32> V: $uint<64>) $out () $is
{
    B0[I][J] := ($bitcast ($uint<64>) V)
}

$module [init_B1_entry]
    $in (I J: $uint<32> V: $uint<64>) $out () $is
{
    B1[I][J] := ($bitcast ($uint<64>) V)
}

$module [init_B2_entry]
    $in (I J: $uint<32> V: $uint<64>) $out () $is
{
    B2[I][J] := ($bitcast ($uint<64>) V)
}

$module [init_B3_entry]
```

```

    $in (I J: $uint<32> V: $uint<64>) $out () $is
{
    B3[I][J] := ($bitcast ($uint<64>) V)
}

// to read the matrices

$module [get_result_entry]
    $in (I J: $uint<32>) $out(V: $int<8>) $is
{
    V := Result[I][J]
}

// Counter to measure time..
$module [counter_daemon] $in () $out () $is
{
    $branchblock[loop] {
        $dopipeline $depth 7 $fullrate
        $merge $entry $loopback
        $phi cval := $zero<32> $on $entry n_cval $on $loopback
        $sendmerge
        $volatile n_cval := (cval + 1)
        GLOBAL_COUNTER := n_cval

        $while 1
    }
}

$module [mmul] $in () $out (nticks: $uint<32>) $is
{
    start_time := GLOBAL_COUNTER
    $branchblock[bb] {
        $merge $entry I_loopback
        $phi I := $zero<8> $on $entry nI $on I_loopback
        $sendmerge
        $volatile nI := (I + 1)

        $merge $entry J_loopback
        $phi J := $zero<8> $on $entry nJ $on J_loopback
        $sendmerge
        $volatile nJ := (J + 1)

        i2 := (I + (N/2))
        j2 := (J + (N/2))

        Result[I][J] := ($call dotp0 (I J))
        Result[I][j2] := ($call dotp1 (I J))
        Result[i2][J] := ($call dotp2 (I J))
        Result[i2][j2] := ($call dotp3 (I J))
    }
}

```

```

        $if (J < {{N/2}} - 1) $then $place [J_loopback] $endif
        $if (I < {{N/2}} - 1) $then $place [I_loopback] $endif
    }
    nticks := (GLOBAL_COUNTER - start_time)
}

$module [dotp0] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
    $branchblock[dotp0] {
        $dopipeline $depth 7 $fullrate

        $merge $entry $loopback
            $phi SUM0 := ($bitcast ($int<8>) 0) $on $entry nSUM0 $on $loop
            $phi SUM1 := ($bitcast ($int<8>) 0) $on $entry nSUM1 $on $loop
            $phi SUM2 := ($bitcast ($int<8>) 0) $on $entry nSUM2 $on $loop
            $phi SUM3 := ($bitcast ($int<8>) 0) $on $entry nSUM3 $on $loop
            $phi K    := $zero<8> $on $entry nK $on $loopback
        $endmerge

        $volatile nK := (K + 2)
        $volatile K_continue := (K < {{N/16}} - 2))

        $volatile K2 := (K + 1)

        nSUM0 := (SUM0 + ($call dot8 (A0[I][K] B0[J][K])))
        nSUM1 := (SUM1 + ($call dot8 (A1[I][K] B2[J][K])))
        nSUM2 := (SUM2 + ($call dot8 (A0[I][K2] B0[J][K2])))
        nSUM3 := (SUM3 + ($call dot8 (A1[I][K2] B2[J][K2])))

        $while K_continue
    } (nSUM0 => nSUM0 nSUM1 => nSUM1 nSUM2 => nSUM2 nSUM3 => nSUM3)

    R := ((nSUM0 + nSUM1) + (nSUM2 + nSUM3))
}

$module [dotp1] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
    $branchblock[dotp1] {
        $dopipeline $depth 7 $fullrate

        $merge $entry $loopback
            $phi SUM0 := ($bitcast ($int<8>) 0) $on $entry nSUM0 $on $loop
            $phi SUM1 := ($bitcast ($int<8>) 0) $on $entry nSUM1 $on $loop
            $phi SUM2 := ($bitcast ($int<8>) 0) $on $entry nSUM2 $on $loop
            $phi SUM3 := ($bitcast ($int<8>) 0) $on $entry nSUM3 $on $loop
            $phi K    := $zero<8> $on $entry nK $on $loopback
        $endmerge

        $volatile nK := (K + 2)
        $volatile K_continue := (K < {{N/16}} - 2))

```



```

    $volatile K2 := (K + 1)

    nSUM0 := (SUM0 + ($call dot8 (A0[I][K] B1[J][K])))
    nSUM1 := (SUM1 + ($call dot8 (A1[I][K] B3[J][K])))
    nSUM2 := (SUM2 + ($call dot8 (A0[I][K2] B1[J][K2])))
    nSUM3 := (SUM3 + ($call dot8 (A1[I][K2] B3[J][K2])))

    $while K_continue
    } (nSUM0 => nSUM0 nSUM1 => nSUM1 nSUM2 => nSUM2 nSUM3 => nSUM3)

    R := ((nSUM0 + nSUM1) + (nSUM2 + nSUM3))
}

$module [dotp2] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
    $branchblock[dotp2] {
        $dopipeline $depth 7 $fullrate

        $merge $entry $loopback
            $phi SUM0 := ($bitcast ($int<8>) 0) $on $entry nSUM0 $on $loop
            $phi SUM1 := ($bitcast ($int<8>) 0) $on $entry nSUM1 $on $loop
            $phi SUM2 := ($bitcast ($int<8>) 0) $on $entry nSUM2 $on $loop
            $phi SUM3 := ($bitcast ($int<8>) 0) $on $entry nSUM3 $on $loop
            $phi K := $zero<8> $on $entry nK $on $loopback
        $endmerge

        $volatile nK := (K + 2)
        $volatile K_continue := (K < {{N/16}-2})

        $volatile K2 := (K + 1)

        nSUM0 := (SUM0 + ($call dot8 (A2[I][K] B0[J][K])))
        nSUM1 := (SUM1 + ($call dot8 (A3[I][K] B2[J][K])))
        nSUM2 := (SUM2 + ($call dot8 (A2[I][K2] B0[J][K2])))
        nSUM3 := (SUM3 + ($call dot8 (A3[I][K2] B2[J][K2])))

        $while K_continue
        } (nSUM0 => nSUM0 nSUM1 => nSUM1 nSUM2 => nSUM2 nSUM3 => nSUM3)

        R := ((nSUM0 + nSUM1) + (nSUM2 + nSUM3))
    }
}

$module [dotp3] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
    $branchblock[dotp3] {
        $dopipeline $depth 7 $fullrate

        $merge $entry $loopback
            $phi SUM0 := ($bitcast ($int<8>) 0) $on $entry nSUM0 $on $loop
            $phi SUM1 := ($bitcast ($int<8>) 0) $on $entry nSUM1 $on $loop
            $phi SUM2 := ($bitcast ($int<8>) 0) $on $entry nSUM2 $on $loop

```

```

    $phi SUM3 := ($bitcast ($int<8>) 0) $on $entry nSUM3 $on $loop
    $phi K     := $zero<8> $on $entry nK $on $loopback
$endmerge

$volatile nK := (K + 2)
$volatile K_continue := (K < {{N/16}-2})

$volatile K2 := (K + 1)

nSUM0 := (SUM0 + ($call dot8 (A2[I][K] B1[J][K])))
nSUM1 := (SUM1 + ($call dot8 (A3[I][K] B3[J][K])))
nSUM2 := (SUM2 + ($call dot8 (A2[I][K2] B1[J][K2])))
nSUM3 := (SUM3 + ($call dot8 (A3[I][K2] B3[J][K2])))

$while K_continue
} (nSUM0 => nSUM0 nSUM1 => nSUM1 nSUM2 => nSUM2 nSUM3 => nSUM3)

R := ((nSUM0 + nSUM1) + (nSUM2 + nSUM3))
}

$pipeline $depth 7 $fullrate $operator
$module[dot8] $in (M N: $uint<64>) $out (R: $int<8>)
$is
{
    // Split into 8 elements (each 8-bit)
    $volatile $split (M 8 8 8 8 8 8 8 8) (M0 M1 M2 M3 M4 M5 M6 M7)
    $volatile $split (N 8 8 8 8 8 8 8 8) (N0 N1 N2 N3 N4 N5 N6 N7)

    // Convert to signed
    $volatile iM0 := ($bitcast ($int<8>) M0)
    $volatile iM1 := ($bitcast ($int<8>) M1)
    $volatile iM2 := ($bitcast ($int<8>) M2)
    $volatile iM3 := ($bitcast ($int<8>) M3)
    $volatile iM4 := ($bitcast ($int<8>) M4)
    $volatile iM5 := ($bitcast ($int<8>) M5)
    $volatile iM6 := ($bitcast ($int<8>) M6)
    $volatile iM7 := ($bitcast ($int<8>) M7)

    $volatile iN0 := ($bitcast ($int<8>) N0)
    $volatile iN1 := ($bitcast ($int<8>) N1)
    $volatile iN2 := ($bitcast ($int<8>) N2)
    $volatile iN3 := ($bitcast ($int<8>) N3)
    $volatile iN4 := ($bitcast ($int<8>) N4)
    $volatile iN5 := ($bitcast ($int<8>) N5)
    $volatile iN6 := ($bitcast ($int<8>) N6)
    $volatile iN7 := ($bitcast ($int<8>) N7)

    // 8 parallel multiplications
    P0 := (iM0 * iN0)
    P1 := (iM1 * iN1)
    P2 := (iM2 * iN2)

```

```

P3 := (iM3 * iN3)
P4 := (iM4 * iN4)
P5 := (iM5 * iN5)
P6 := (iM6 * iN6)
P7 := (iM7 * iN7)

// Adder tree (log depth reduction)
S0 := (P0 + P1)
S1 := (P2 + P3)
S2 := (P4 + P5)
S3 := (P6 + P7)

T0 := (S0 + S1)
T1 := (S2 + S3)

R := (T0 + T1)
}

```

6 Testing

6.1 Test 1 : $A \times I$

A matrix is such that $A[x][y] = x + y$ and I is identity matrix which results as output A

```
[INFO] Results fetched successfully.
[INFO] Computation finished. Ticks = 65633

----- MATRIX A -----
  0   1   2   3   4   5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46
47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63
  1   2   3   4   5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47
48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64
  2   3   4   5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48
49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65
  3   4   5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49
50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66
  4   5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50
51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67
  5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51
52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68
  6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52
53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69
  7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53
54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70
  8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55
56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71
  9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56
57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
 10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56
57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73
 11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56
```

Figure 2: matrix A

[illegible]

Figure 3: matrix B

		RESULT MATRIX																																																												
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
58	59	60	61	62	63																																																									
59	60	61	62	63	64	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40 <td>41</td> <td>42</td> <td>43</td> <td>44</td> <td>45<td>46</td><td>47</td><td>48</td><td>49</td><td>50</td><td>51</td><td>52</td><td>53</td><td>54</td><td>55</td><td>56</td><td>57</td><td>58</td><td>59</td><td>60</td></td>	41	42	43	44	45 <td>46</td> <td>47</td> <td>48</td> <td>49</td> <td>50</td> <td>51</td> <td>52</td> <td>53</td> <td>54</td> <td>55</td> <td>56</td> <td>57</td> <td>58</td> <td>59</td> <td>60</td>	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60			
60	61	62	63	64	65	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40 <td>41</td> <td>42</td> <td>43</td> <td>44</td> <td>45<td>46</td><td>47</td><td>48</td><td>49</td><td>50</td><td>51</td><td>52</td><td>53</td><td>54</td><td>55</td><td>56</td><td>57</td><td>58</td><td>59</td><td>60</td></td>	41	42	43	44	45 <td>46</td> <td>47</td> <td>48</td> <td>49</td> <td>50</td> <td>51</td> <td>52</td> <td>53</td> <td>54</td> <td>55</td> <td>56</td> <td>57</td> <td>58</td> <td>59</td> <td>60</td>	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60			
61	62	63	64	65	66	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40 <td>41</td> <td>42</td> <td>43</td> <td>44</td> <td>45<td>46</td><td>47</td><td>48</td><td>49</td><td>50</td><td>51</td><td>52</td><td>53</td><td>54</td><td>55</td><td>56</td><td>57</td><td>58</td><td>59</td><td>60</td></td>	41	42	43	44	45 <td>46</td> <td>47</td> <td>48</td> <td>49</td> <td>50</td> <td>51</td> <td>52</td> <td>53</td> <td>54</td> <td>55</td> <td>56</td> <td>57</td> <td>58</td> <td>59</td> <td>60</td>	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60			
62	63	64	65	66	67	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40 <td>41</td> <td>42</td> <td>43</td> <td>44</td> <td>45<td>46</td><td>47</td><td>48</td><td>49</td><td>50</td><td>51</td><td>52</td><td>53</td><td>54</td><td>55</td><td>56</td><td>57</td><td>58</td><td>59</td><td>60</td></td>	41	42	43	44	45 <td>46</td> <td>47</td> <td>48</td> <td>49</td> <td>50</td> <td>51</td> <td>52</td> <td>53</td> <td>54</td> <td>55</td> <td>56</td> <td>57</td> <td>58</td> <td>59</td> <td>60</td>	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60			
63	64	65	66	67	68	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40 <td>41</td> <td>42</td> <td>43</td> <td>44</td> <td>45<td>46</td><td>47</td><td>48</td><td>49</td><td>50</td><td>51</td><td>52</td><td>53</td><td>54</td><td>55</td><td>56</td><td>57</td><td>58</td><td>59</td><td>60</td></td>	41	42	43	44	45 <td>46</td> <td>47</td> <td>48</td> <td>49</td> <td>50</td> <td>51</td> <td>52</td> <td>53</td> <td>54</td> <td>55</td> <td>56</td> <td>57</td> <td>58</td> <td>59</td> <td>60</td>	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60			
64	65	66	67	68	69	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40 <td>41</td> <td>42</td> <td>43</td> <td>44</td> <td>45<td>46</td><td>47</td><td>48</td><td>49</td><td>50</td><td>51</td><td>52</td><td>53</td><td>54</td><td>55</td><td>56</td><td>57</td><td>58</td><td>59</td><td>60</td></td>	41	42	43	44	45 <td>46</td> <td>47</td> <td>48</td> <td>49</td> <td>50</td> <td>51</td> <td>52</td> <td>53</td> <td>54</td> <td>55</td> <td>56</td> <td>57</td> <td>58</td> <td>59</td> <td>60</td>	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60			
65	66	67	68	69	70	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40 <td>41</td> <td>42</td> <td>43</td> <td>44</td> <td>45<td>46</td><td>47</td><td>48</td><td>49</td><td>50</td><td>51</td><td>52</td><td>53</td><td>54</td><td>55</td><td>56</td><td>57</td><td>58</td><td>59</td><td>60</td></td>	41	42	43	44	45 <td>46</td> <td>47</td> <td>48</td> <td>49</td> <td>50</td> <td>51</td> <td>52</td> <td>53</td> <td>54</td> <td>55</td> <td>56</td> <td>57</td> <td>58</td> <td>59</td> <td>60</td>	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60			
66	67	68	69	70	71	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40 <td>41</td> <td>42</td> <td>43</td> <td>44</td> <td>45<td>46</td><td>47</td><td>48</td><td>49</td><td>50</td><td>51</td><td>52</td><td>53</td><td>54</td><td>55</td><td>56</td><td>57</td><td>58</td><td>59</td><td>60</td></td>	41	42	43	44	45 <td>46</td> <td>47</td> <td>48</td> <td>49</td> <td>50</td> <td>51</td> <td>52</td> <td>53</td> <td>54</td> <td>55</td> <td>56</td> <td>57</td> <td>58</td> <td>59</td> <td>60</td>	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60			
67	68	69	70	71	72	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31</																																

Figure 4: result matrix

Note: For the The computed values are clipped to 8-bit signed representation. Hence, overflow leads to wrapping, which results in negative values appearing in the output.

6.2 Test 2 : $A \times B$

A matrix is such that $A[x][y] = x$ and $B[x][y] = y$

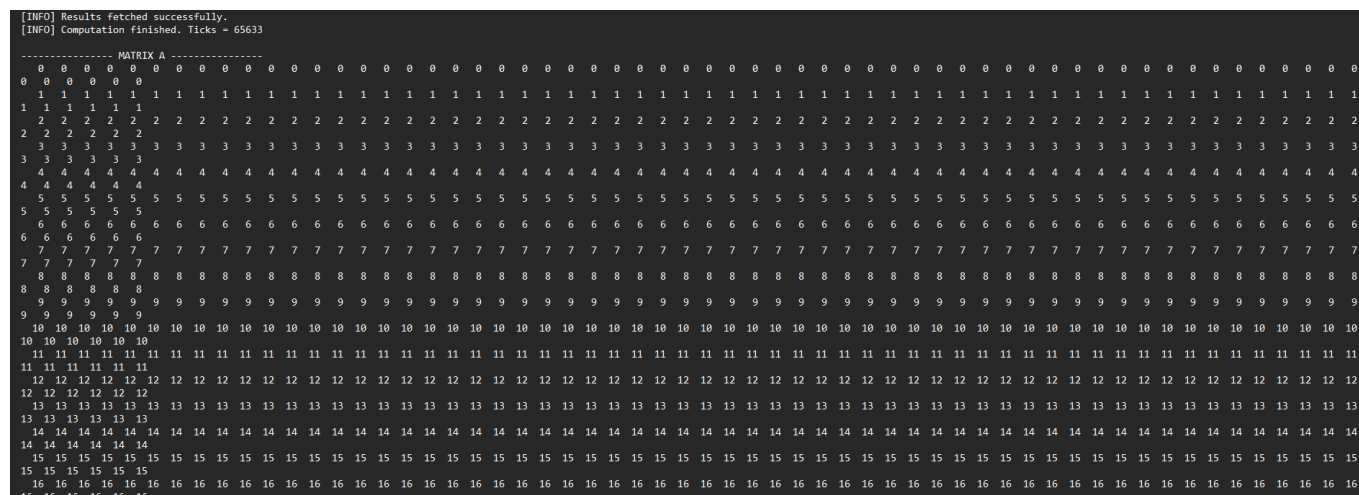


Figure 5: matrix A

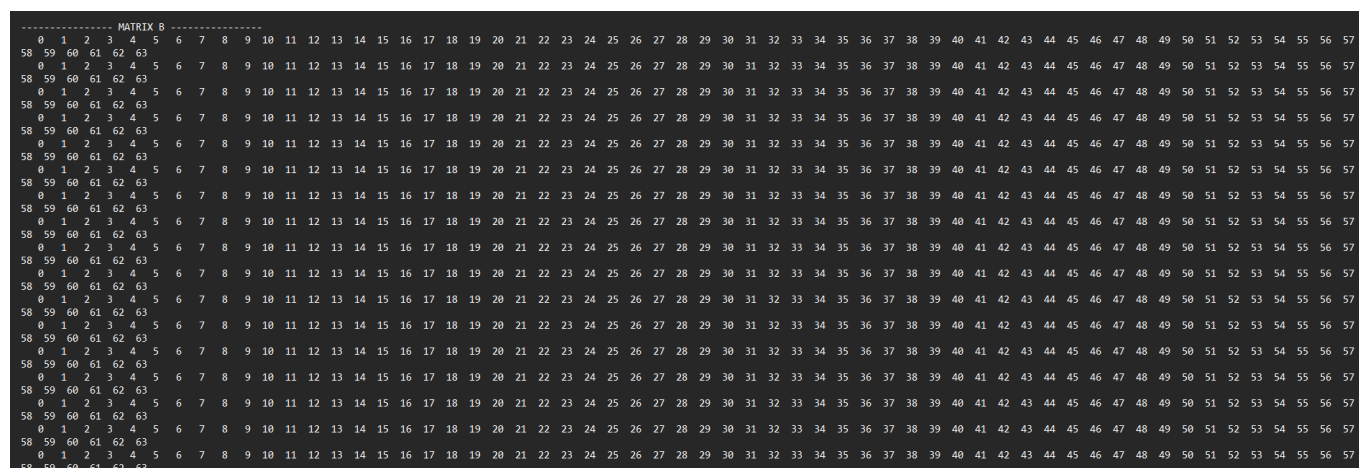


Figure 6: matrix B

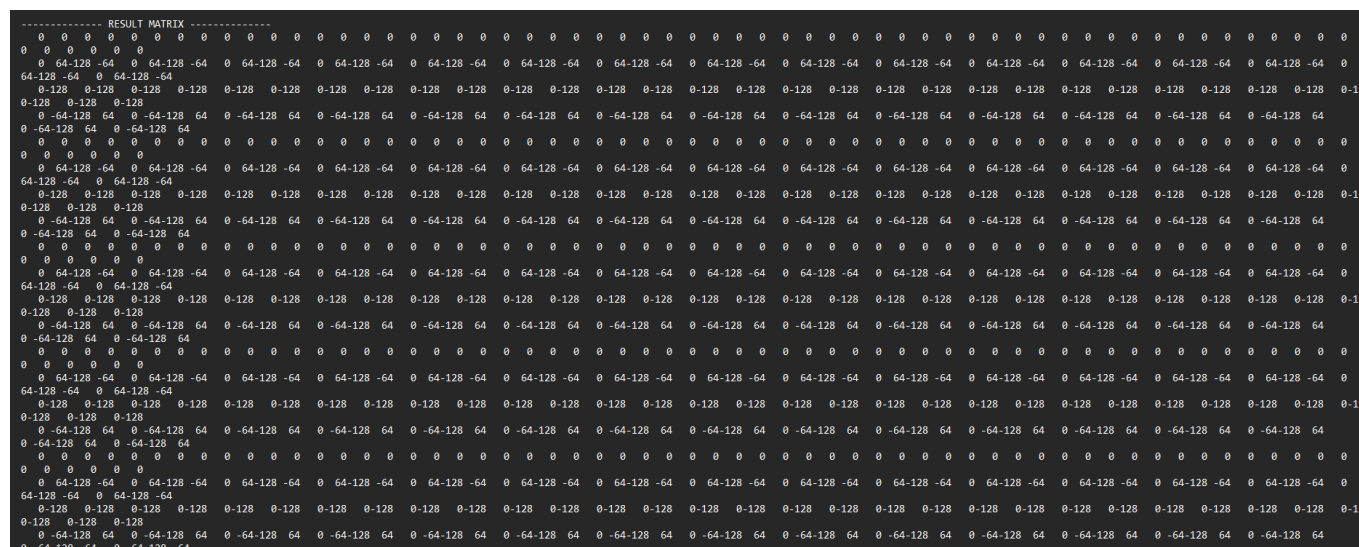


Figure 7: result matrix

6.3 Test 3 : $A \times B$

A matrix is such that $A[x][y] = x \bmod 4$ and $B[x][y] = y \bmod 4$

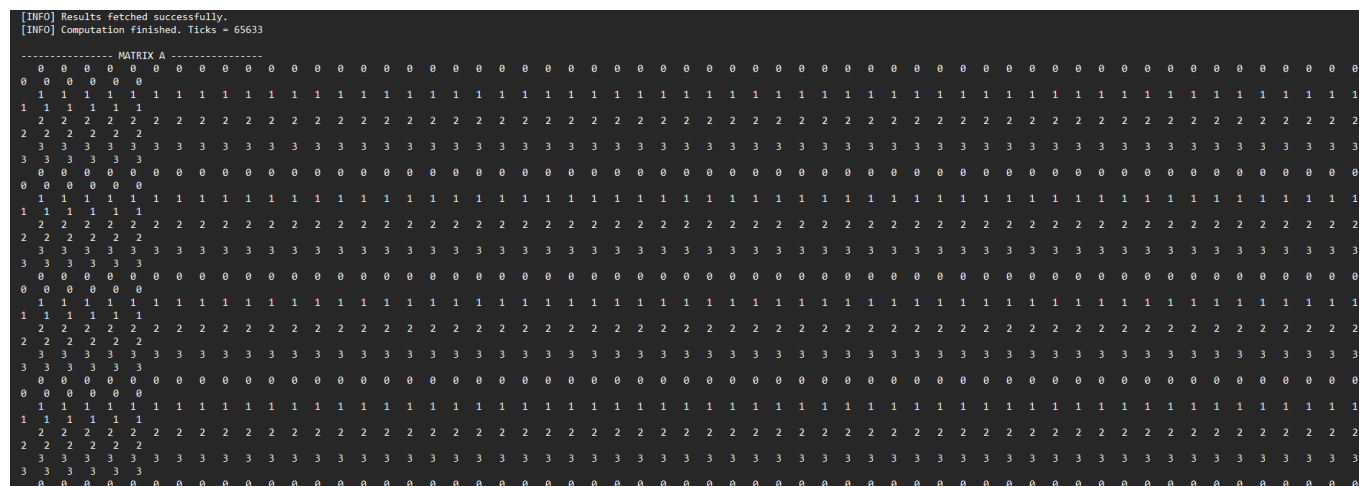


Figure 8: matrix A

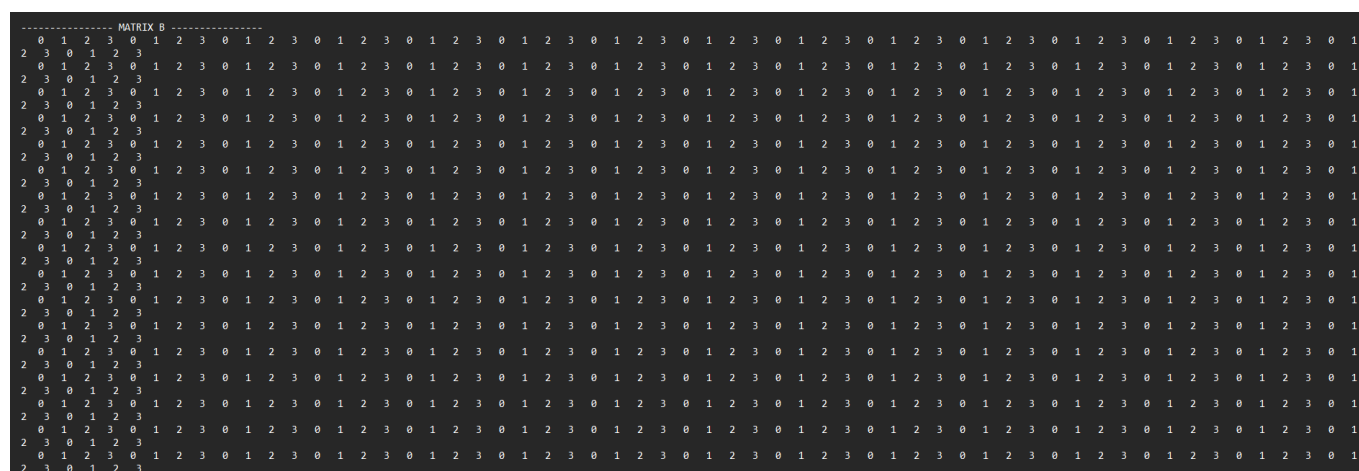


Figure 9: matrix B

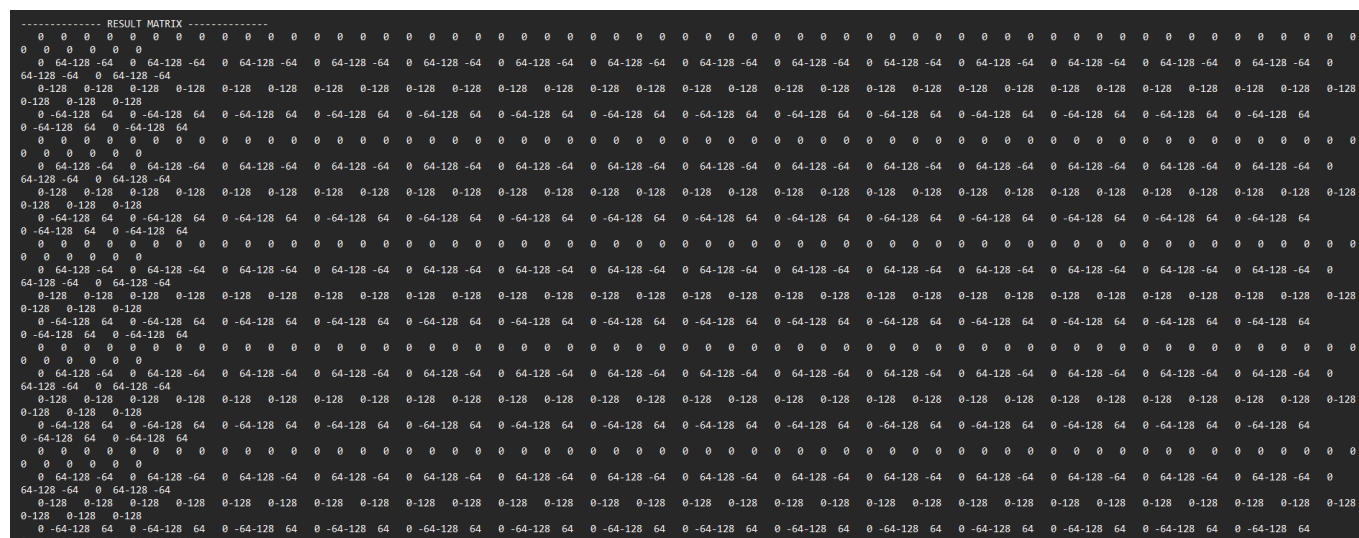


Figure 10: result matrix

6.4 Test 4 : $A \times B$

A matrix is such that $A[x][y] = x \bmod 4$ and $B[x][y] = x \bmod 4$

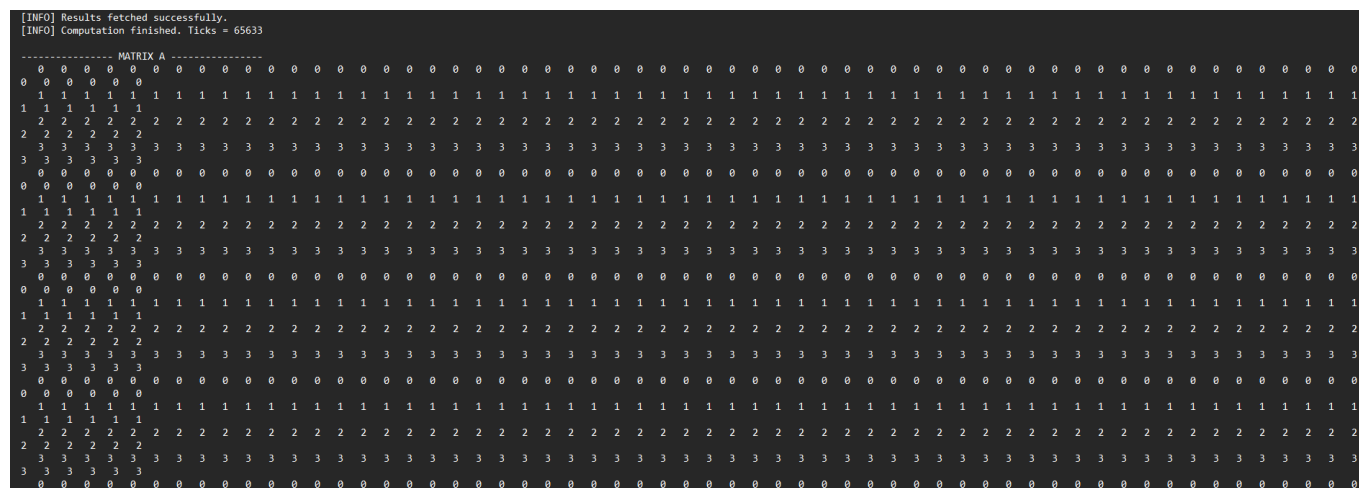


Figure 11: matrix A

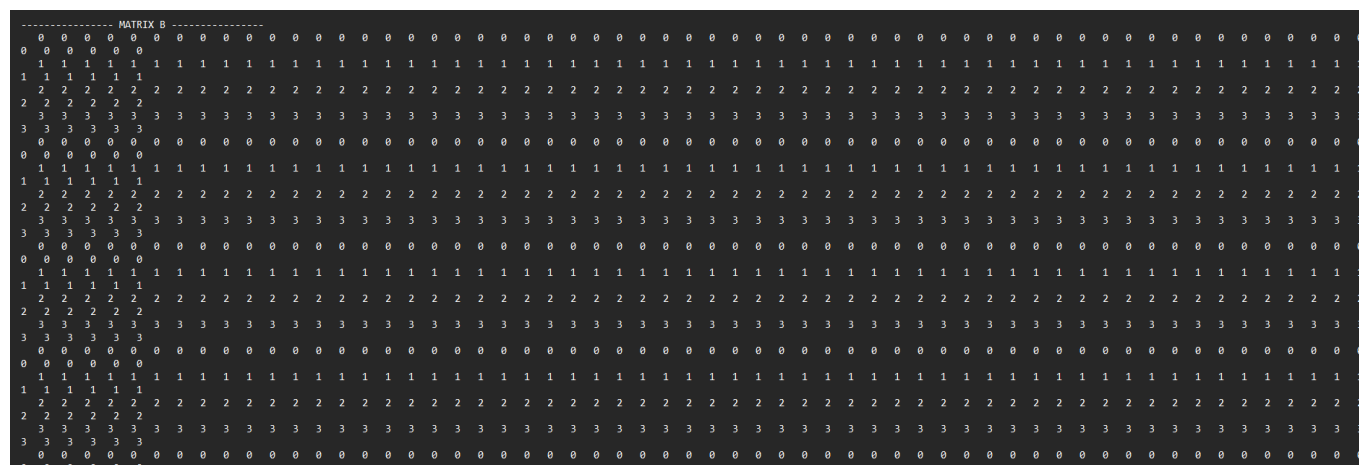


Figure 12: matrix B

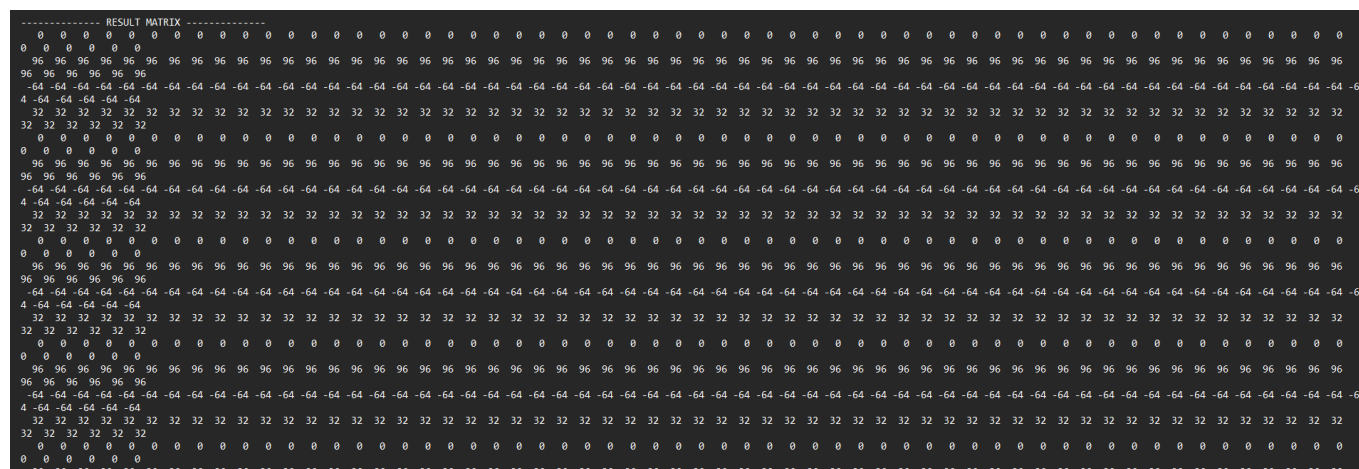


Figure 13: result matrix